

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORY 2

Meet the OS You Will Build

Done by: st. gr. FAF-241, Lucian-Adrian Gavril

Checked by: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3

INTRODUCTION 4

1 COMPILATION, BOOTING, AND SHELL OPERATIONS 5

1.1 Compilation Benchmark and Boot Sequence 5

1.2 Interactive Shell Commands and Pipeline Execution 5

1.3 Answers to Observe Prompts (Part 2: xv6 Shell) 6

2 SOURCE CODE INSPECTION: USER VS. KERNEL BOUNDARY 7

2.1 System Call Interfaces in user/cat.c 7

2.2 System Call Dispatch Path 7

2.3 Answers to Observe Prompts (Part 3: Reading Source) 7

3 USER UTILITIES: SLEEP AND PINGPONG 8

3.1 Implementation of user/sleep.c 8

3.2 Build System Registration and Verification 8

3.3 Optional Two-Pipe Communication: pingpong.c 8

CONCLUSIONS 10

ABSTRACT

This laboratory report documents the compilation, booting, source-code analysis, and user-space program development for the MIT xv6 educational operating system running on an emulated RISC-V platform via QEMU. The experimental work explores the architectural boundary between unprivileged user processes and privileged kernel services. A new user utility, `sleep.c`, was implemented with strict argument validation and registered in the xv6 build system under `UPROGS`. Furthermore, an inter-process communication exchange utilizing two unidirectional pipes was authored and verified in `pingpong.c`. Build benchmarks, boot sequences, and command outputs confirm the correct operation of system call dispatching, pipeline coordination, and scheduler blocking.

Keywords: operating systems, xv6, riscv, system calls, qemu, process scheduling, pipelines

INTRODUCTION

While modern production operating systems such as Linux comprise millions of lines of source code, educational kernels provide transparent environments where kernel mechanisms can be studied directly. The MIT xv6 operating system is a modern re-implementation of Unix Version 6 written in approximately 9,000 lines of C for the RISC-V 64-bit architecture.

The objective of Laboratory 2 is to build xv6 from source, execute it within the QEMU emulator, analyze its shell and user/kernel separation, implement a user-space utility that triggers kernel system calls, and examine inter-process pipe communication:

a) **Build and Emulation**: Compiling the RISC-V kernel, formatting the initial file system image (`fs.img`), and booting the system to the initial shell prompt;

b) **Shell Operations**: Evaluating primitive Unix commands (`ls`, `cat`, `echo`, `wc`) and analyzing how pipelines connect child processes via kernel buffers;

c) **Codebase Architecture**: Tracing user-space library stubs, assembly trap entries (`ecall`), system call dispatch tables, and privileged kernel handlers;

d) **User Program Development**: Authoring `user/sleep.c` to interface with the kernel timer service and authoring `pingpong.c` for bidirectional pipe communication.

1 COMPILATION, BOOTING, AND SHELL OPERATIONS

1.1 Compilation Benchmark and Boot Sequence

The kernel and user utilities were compiled with the RISC-V cross-compiler toolchain, and the filesystem image was generated using `mkfs`. Compilation completed in 2.158 seconds. The guest kernel booted under QEMU 8.2.2 and presented the interactive shell prompt after 2.459 seconds:

```
$ make -C xv6-riscv -j2 kernel/kernel fs.img
Build elapsed seconds: 2.158

$ qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel \
  -m 128M -smp 3 -nographic -drive file=fs.img,if=none,format=raw,id=x0 \
  -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$
```

The `init: starting sh` line marks the successful transition from kernel hardware initialization to the execution of the first user-space process (PID 1).

1.2 Interactive Shell Commands and Pipeline Execution

Standard utilities were executed to confirm filesystem mounting and executable loading:

```
$ ls
README          2 2 2425
cat             2 3 35176
echo            2 4 34096
grep            2 6 42520
sleep           2 14 33824
wc              2 18 36128
console         3 23 0

$ cat README | head -n 2
```

```
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6). xv6 loosely follows the structure and style of v6,

$ echo hello xv6
hello xv6

$ ls | grep c
cat             2 3 35176
echo            2 4 34096
wc              2 18 36128
console         3 23 0

$ wc README
48 334 2425 README
```

1.3 Answers to Observe Prompts (Part 2: xv6 Shell)

a) **Included Programs**: Three programs compiled into the filesystem image and listed in `UPROGS` are `ls`, `cat`, and `echo`;

b) **Pipeline Coordination**: The command `ls | grep c` requires two separate processes (one producer and one consumer) because each binary executes an independent instruction stream and needs private file descriptors. In `user/sh.c`, the shell allocates a pipe via `pipe()`, forks child processes, redirects file descriptors using `dup()` and `close()`, and lets the kernel transfer data through a circular memory buffer;

c) **Comparison with Linux Shell**: The xv6 shell provides a minimal set of control structures (pipes, sequential execution, redirection, background jobs) without environment variable parsing, globbing, or terminal readline history. The Linux shell provides complete POSIX expansions, job control, and extensive script grammar.

2 SOURCE CODE INSPECTION: USER VS. KERNEL BOUNDARY

2.1 System Call Interfaces in user/cat.c

Inspecting `user/cat.c` reveals the explicit boundary between application logic and kernel services:

- `open(argv[i], O_RDONLY)`: Requests the kernel to allocate a file descriptor;
- `read(fd, buf, sizeof(buf))`: Traps into the kernel to fetch data blocks from disk;
- `write(1, buf, n)`: Transmits bytes to standard output (file descriptor 1);
- `close(fd)`: Releases the allocated file descriptor in the process table;
- `exit(0)`: Terminates process execution and yields exit status to the parent.

While `fprintf` formats text in user space via `user/printf.c`, it ultimately invokes the `write()` system call to display characters.

2.2 System Call Dispatch Path

Table 2.1 traces the execution trajectory of a user request from application space to physical execution in kernel mode.

Table 2.1 – System call dispatch trajectory in xv6

Layer	Source File	Role in Execution
User Source	`user/sleep.c`	Validates command-line parameters and invokes the stub.
User Interface	`user/user.h`	Prototypes function signature for the C compiler.
Assembly Stub	`user/usys.S`	Loads system call number into register `a7` and issues `ecall`.
Trap Handler	`kernel/trap.c`	Saves CPU context and routes trap to `syscall()`.
Dispatcher	`kernel/syscall.c`	Indexes `syscalls[]` array by number in `a7`.
Kernel Service	`kernel/sysproc.c`	Implements privileged handler (`sys_sleep` / `sys_pause`).

2.3 Answers to Observe Prompts (Part 3: Reading Source)

a) **System Calls in cat.c**: The utility uses `open`, `read`, `write`, `close`, and `exit`. The formatted output routine `fprintf` is a user-space library function;

b) **Implementation of sys_read**: The handler `sys_read` is implemented in `kernel/sysfile.c` (line 69);

c) **User vs. Kernel Distinction**: Code under `kernel/` executes in RISC-V supervisor mode with unrestricted access to physical memory, page tables, and hardware device registers. Code under `user/` executes in unprivileged user mode and cannot access hardware directly; it must request privileged operations through system calls.

3 USER UTILITIES: SLEEP AND PINGPONG

3.1 Implementation of user/sleep.c

The `sleep` utility was authored to accept a positive tick duration:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc != 2) {
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
```

In accordance with current course xv6 specifications, the program delegates to the kernel tick delay interface (`pause()`).

3.2 Build System Registration and Verification

The program was registered in the `Makefile` by appending `$U/_sleep\` to `UPROGS`. Live execution under QEMU verified both normal and error paths:

```
$ sleep 10
# Returns cleanly to prompt after 0.958 seconds wall-clock time

$ sleep
usage: sleep <ticks>
```

3.3 Optional Two-Pipe Communication: pingpong.c

To verify bidirectional inter-process communication without race conditions, `pingpong.c` establishes two separate unidirectional pipes before calling `fork()`:

```
$ pingpong
```

```
4: received ping
3: received pong
```

The parent process writes a token byte to `parent_to_child[1]`, which the child (PID 4) reads and announces. The child then writes the byte back to `child_to_parent[1]`, and the parent (PID 3) completes the exchange and reaps the child process. Closing unused pipe descriptor copies ensures proper EOF signaling.

CONCLUSIONS

Laboratory 2 provided hands-on experience in building, booting, and extending the xv6 Unix operating system. The experimental work established that:

a) Operating system hardware abstractions decouple user applications from raw CPU architecture;
b) System calls enforce security and isolation by transitioning the processor from unprivileged user mode to supervisor mode via defined assembly stubs;
c) Interprocess pipelines rely on kernel-mediated buffer synchronization to coordinate concurrent producer and consumer processes;
d) Registering user binaries in `UPROGS` integrates compiled programs directly into the bootable filesystem image.